

EECE 2140

Computing Fundamentals for Engineers

System Design Overview

Word Search Puzzle Generator and Solver

A C++ Engineering Simulation Project

 **Team Members:** Matthew Rondon

 **Instructor:** Dr. Fatema Nafa

Semester: Spring 2026

 **Repository:** <https://github.com/rondonmatthew>

Contents

1	Project Overview	2
1.1	Motivation	2
1.2	Project Goals	2
2	Basic Functionalities	3
2.1	Functionality Summary	3
2.2	Detailed Functionality	3
3	Object-Oriented Programming (OOP) Design	4
3.1	Class Relationship Overview	4
3.2	OOP Design Table	5
3.3	Key OOP Requirements	5
4	System Overview — Input, Process, Output	5
4.1	High-Level Description	5
4.2	IPO Diagram	6
4.3	Detailed Step Description	6
5	Pseudo Code	6
5.1	Pseudocode	7
6	C++ Implementation Highlights	7
7	Challenges, Solutions & Lessons Learned	8
7.1	Challenges & Solutions	8
7.2	Lessons Learned	8
8	Limitations & Future Work	9
9	GitHub Repository Structure	9
A	Sample Program Output	9

1 Project Overview

💡 Project Description

The **Word Search Puzzle Generator and Solver** is a C++ program that creates and solves word search puzzles using a two-dimensional grid. The program places words into the grid using backtracking and then searches the puzzle to find each word and its location.

This project was designed to apply programming concepts learned throughout the course, including vectors, functions, object-oriented programming, recursion, and modular design. The project also includes simple performance tracking to measure generation and solving time.

1.1 Motivation

Growing up I loved working on word search puzzles with my family. trying to find words faster than my family members made me love friendly competition. Bringing that love for word searches to my final project provided a simple but effective way to apply the core programming concepts learned in class while bringing out my inner child into this project.

The project uses a two-dimensional grid to represent the puzzle and incorporates classroom techniques such as functions, classes, object-oriented programming, and the use of vectors to store and manage both the grid and the word list.

In addition, the project includes backtracking. This concept is essential for solving more complex problems by allowing the program to delete and rethink positions when needed.

1.2 Project Goals

The main goal of this project was to design and implement a word search puzzle generator and solver using C++. The project focused on they key functionality and implementation concepts we learned throughout the semester.

Functionality Goals:

- Generate a word search puzzle from a list of inputted words
- Place words in multiple(8) directions, including horizontal, vertical, and diagonal
- Develop a solver that can locate words within the grid

Implementation Goals:

- Use object-oriented programming (OOP) to organize the program into classes
- Use vectors and references to manage data
- Use backtracking to correctly place words in the grid
- Ensure valid word placement while allowing overlapping letters when needed

Measurable Goals:

- Track and analyze performance of the program
- Track generation and solving times
- Count the number of backtracking attempts during program

2 Basic Functionalities

This section lists all core functionalities of the program together with their expected inputs, outputs, and contribution to the overall system.

2.1 Functionality Summary

The program consists of two main components: a puzzle generator and a puzzle solver.

The puzzle generator creates a word search grid and places words into the grid. It begins by initializing an empty two-dimensional grid and then attempts to place each word. Words can be placed in a total of eight directions, including horizontal, vertical, and diagonal. The generator ensures that all placements are valid by checking boundaries and avoiding conflicting letters. If a word cannot be placed, the program uses backtracking to remove the word and tries a new position.

The puzzle solver finds the words within the generated grid. It scans the grid and checks each position for the first letter of a word. If a match is found, it searches in all possible directions to determine if the full word exists. When a word is found, the program records its starting position and direction.

In addition, the program measures generation time, solving time, and the number of backtracking steps. This allows for evaluation of the program's effectiveness under different starting conditions.

2.2 Detailed Functionality

The program is divided into multiple components that work together to generate and solve a word search puzzle.

The first step is initializing the grid. A two-dimensional vector is used to represent the puzzle, and each position is initially filled with a random character to represent an empty space.

The puzzle generator then processes a list of input words. For each word, the program attempts to place it into the grid by checking all possible positions and directions. The placement must stay within the bounds of the grid and cannot overwrite letters unless they match. This allows the possibility of the overlapping of words.

If a word cannot be placed successfully, the program uses a backtracking approach. This means it removes previously placed words and tries different positions until a valid position

is found or all options are exercised.

Once all words are placed, any remaining empty spaces in the grid are filled with random letters to complete the puzzle.

After the puzzle is generated, the solver begins its process. It scans each cell in the grid and checks if the first letter matches a word from the list. If a match is found, the solver checks all eight possible directions to determine if the full word exists.

When a word is successfully found, the program records its starting position and direction. The measured data is then displayed as part of the output.

Finally, the performance analyzer records how long it takes to generate and solve the puzzle, as well as the number of backtracking steps during the generation.

3 Object-Oriented Programming (OOP) Design

3.1 Class Relationship Overview

This program is designed using object-oriented programming to separate different parts of the system into manageable components. Each class is responsible for a specific part of the program, which creates a well organized programming setup.

The **Grid** class is used to represent the two-dimensional puzzle. It stores the letters and provides functions to initialize, display, and modify the grid.

The **PuzzleGenerator** class is responsible for placing words into the grid. It uses the Grid class to access and modify the puzzle while applying backtracking to ensure correct placement.

The **PuzzleSolver** class searches the grid to locate each word. It checks each position and explores all possible directions to find matches.

The **PerformanceAnalyzer** class is used to measure how long different parts of the program take to run. It tracks generation time, solving time, and helps evaluate effectiveness.

These classes work together by sharing the grid and word list, allowing the generator to create the puzzle and the solver to analyze it.

3.2 OOP Design Table

Class	Purpose	Main Functions
Grid	Stores and manages the 2D word search grid.	initialize(), display(), setLetter(), getLetter(), isInBounds(), fillEmptySpaces()
PuzzleGenerator	Places words into the grid using valid placement checks and backtracking.	generate(), placeWords(), canPlaceWord(), placeWord(), removeWord()
PuzzleSolver	Searches the completed grid to find each word.	solve(), searchDirection()
PerformanceAnalyzer	Measures how long the generator and solver take to run.	startTimer(), stopTimer(), getElapsedTime()

3.3 Key OOP Requirements

This project applies several key object-oriented programming concepts learned throughout the course.

Encapsulation is used by organizing the program into separate classes, each responsible for a specific task such as managing the grid, generating the puzzle, solving the puzzle, and analyzing performance.

Abstraction is applied by allowing each class to handle its own internal logic while providing simple functions that can be used by other parts of the program.

Modularity is achieved by breaking the program into multiple files and classes, making the code easier to read, maintain, and debug.

The use of references allows efficient data sharing between classes without unnecessary copying, especially when passing the grid between the generator and solver.

4 System Overview — Input, Process, Output

4.1 High-Level Description

The program follows a simple Input, Process, Output (IPO) model.

The input to the program consists of a list of words and the desired grid size. These values are provided at the start of the program.

The process begins with the puzzle generator, which attempts to place each word into the grid. It checks all possible positions and directions while ensuring that each placement is valid. If a placement fails, the program uses backtracking to remove previously placed words and try new positions.

After the puzzle is generated, the solver scans the grid to find each word. It checks every position and searches in all possible directions to locate matches.

The output of the program includes the completed word search puzzle, the locations of each word, and performance results such as generation time, solving time, and backtracking steps.

4.2 IPO Diagram

Input	Process	Output
Word list	Generate puzzle using backtracking	Completed word search puzzle
Grid size	Store letters in 2D vector grid	Word starting positions
Words to search	Solver scans grid in all directions	Word directions
Program execution	Measure runtime and backtracking steps	Performance results

4.3 Detailed Step Description

The algorithm begins by attempting to place words into the grid one at a time. The program starts with the first word in the list and tries to place it in every possible position and direction within the grid.

For each position, the program checks if the word can fit within the boundaries of the grid. It also ensures that the placement does not overwrite existing letters unless they match, allowing valid overlaps between words.

If a valid placement is found, the word is placed into the grid. The program then moves on to the next word and repeats the process.

If the program reaches a point where a word cannot be placed, it uses backtracking. This means it removes the previously placed word and tries a different position or direction. This process continues until all words are successfully placed or no valid solution can be found.

Once all words are placed, the program fills any remaining empty spaces with random letters to complete the puzzle.

After the puzzle is generated, the solver begins scanning the grid. It checks each cell for the first letter of a word and then searches in all directions to determine if the full word exists. When a word is found, the program records its position and direction.

5 Pseudo Code

5.1 Pseudocode

The main algorithm used in this project is a backtracking approach for placing words into the grid.

```
function placeWords(index):
    if index == totalWords:
        return true

    word = words[index]

    for each row in grid:
        for each column in grid:
            for each direction:
                if placement is valid:
                    place word

                    if placeWords(index + 1):
                        return true

            remove word // backtrack

    return false
```

The solver uses a search-based approach:

```
for each word:
    for each cell in grid:
        if first letter matches:
            check all directions:
                if full word matches:
                    record position and direction
```

6 C++ Implementation Highlights

This project was implemented using multiple C++ files to keep the program organized. Each class has its own header file and source file. This made the code easier to understand, test, and debug.

The **Grid** class uses a two-dimensional vector to store the puzzle:

```
vector<vector<char>>> grid;
```

This allows the program to store letters in rows and columns.

The **PuzzleGenerator** class is responsible for placing words into the grid. It checks whether each word can be placed in a valid position before adding it to the puzzle. If a word placement causes a problem later, the program uses backtracking to remove the word and try another position.

The **PuzzleSolver** class searches the completed puzzle. It checks every cell in the grid

and searches in all possible directions when the first letter of a word is found.

The **PerformanceAnalyzer** class uses simple timing functions to measure how long the generator and solver take to run. This helped compare performance using different grid sizes and word counts.

Overall, the implementation focuses on class organization, vectors, loops, functions, and recursion, which were all important concepts from the course.

7 Challenges, Solutions & Lessons Learned

7.1 Challenges & Solutions

During this project, several challenges were encountered while developing the word search generator and solver.

One of the main challenges was ensuring that words were placed in the grid in a valid orientation. This required checking that each word stayed within the boundaries of the grid while also allowing placement in all eight directions.

Another challenge was implementing backtracking. It was initially difficult to understand how to remove a previously placed word and reposition it when a placement caused a conflict later in the process. This was resolved by breaking the problem into smaller steps and carefully tracing how the algorithm moved forward and backward.

A specific issue occurred when trying to keep word placements within the grid boundaries. Some placements would go out of bounds, causing errors. To solve this, a boundary checking function was used to verify that each letter of the word stayed within the grid before placing it.

To overcome these challenges, the problems were approached step by step instead of all at once. When a solution was difficult to understand, additional resources were used to better understand the correct approach and apply it to the program.

7.2 Lessons Learned

This project helped strengthen my understanding of several important programming concepts. I gained more experience working with vectors and two-dimensional data structures.

I also learned how backtracking works in practice and how recursion can be used to solve problems that require trying multiple possibilities.

In addition, I improved my ability to break down complex problems into smaller steps. This made it easier to debug the program and understand how each part worked together.

Overall, this project helped me become more comfortable with object-oriented programming and organizing code into separate classes and files.

8 Limitations & Future Work

One limitation of this program is that words tend to be placed starting from the top of the grid. This happens because the program checks positions in order and places words in the first valid location it finds.

Another limitation is that the word list is currently hardcoded into the program instead of being read from user input or a file.

If this project were to be improved, one possible enhancement would be to randomize the starting positions and directions when placing words. This would help distribute words more evenly throughout the grid.

Additional improvements could include adding a graphical user interface (GUI) or allowing the user to input custom word lists and grid sizes.

9 GitHub Repository Structure

</> Repository Layout

```
WordSearchProject/  
|-- main.cpp  
|-- Grid.h  
|-- Grid.cpp  
|-- PuzzleGenerator.h  
|-- PuzzleGenerator.cpp  
|-- PuzzleSolver.h  
|-- PuzzleSolver.cpp  
|-- PerformanceAnalyzer.h  
|-- PerformanceAnalyzer.cpp  
|-- README.md
```

A Sample Program Output

 Expected Console Output - End-of-Run Report

Generated Puzzle:

```
B A C K G H O S T F
O J L O I K N P O S
Y V A L D Z C P X P
H H S Y D E B G I M
Q J S N N K U Q N D
C O J N B L X K F Q
B V S N Y X B G D Q
P A S P A V T G E Q
M S Z V A A F Y U C
X Y G K C H R V D Z
```

Solver Results:

```
BACK found at (0, 0) Direction: Right
CLASS found at (0, 2) Direction: Down
GHOST found at (0, 4) Direction: Right
CODE found at (0, 2) Direction: Down Right
```

Performance Results:

```
Generation Time: 2e-05 seconds
Solving Time: 5.5e-05 seconds
Backtracking Steps: 0
```